



Tipus bàsics de dades en Python

- `int` : nombres enters
- `float` : nombres reals amb decimals
- `str` : cadenes de text
- `bool` : valors booleans (`True` o `False`)
- `bytes` : dades binàries

```
In [ ]: n = 12345
print(n.bit_length())           # Nombre de bits per representar l'enter
print(n.as_integer_ratio())      # Tuple (numerador, denominador)
```

```
f = 12.75
print(f.is_integer())           # False, perquè té decimals
print(f.as_integer_ratio())     # (51, 4)
```

Variables i convencions

Python és un **llenguatge no tipat** i per tant no cal declarar-ne el tipus.
Concretament, Python és un llenguatge **no tipat dinàmic** perquè una mateixa variable pot guardar diferents tipus de dades en moments diferents, i que el sistema gestiona automàticament el tipus segons el valor que s'hi assigna.

```
In [ ]: nom = "Anna"
        edat = 23
        preu_total = 12.5
```

Python assigna el tipus automàticament segons el valor assignat a la variable.

Bones pràctiques i convencions de noms de variables:

- Els noms de les variables han de començar per lletra o _
- Són sensibles a majúscules/minúscules
- Només lletres, dígitos i _
- No poden ser paraules reservades (if, else,...)
- És recomanable fer servir "snake_case" per variables (nombre_alumnes, dies_setmana)

Comentaris del codi

Els comentaris es fan amb el símbol # :

```
In [ ]: # Aquest és un comentari de línia
        preu = 10 # Preu sense descompte
```

Operadors matemàtics i precedència

Els principals operadors matemàtics són:

Operador	Significat	Exemple
+	suma	2 + 3
-	resta	5 - 2
*	multiplicació	4 * 7
/	divisió	8 / 2
//	divisió sencera	7 // 2
%	residu de la divisió	9 % 2
**	potència	2 ** 3

La precedència dels operadors de més a menys important és:

1. Parèntesis ()

2. Potències `**`
3. Negació `-x` i `+x`
4. Multiplicació, divisió, residu `*` `/` `//` `%`
5. Suma i resta `+` `-`
6. Comparadors (`==`, `>`, `in`, ...))
7. Lògics (`not`, `and`, `or`).

Proposta d'exercici: posa els parèntesis perquè el resultat sigui igual.

```
In [ ]: 3 * 5 + 7 / 6 **3
```

Conversió de tipus (*casting*)

És el procés pel qual transformem un valor d'un tipus de dada a un altre.

Per exemple:

- `int()` converteix un valor a enter.
- `float()` a nombre decimal
- `str()` a cadena de text
- `bool()` a valor booleà

```
In [ ]: d = "123"           # d és un string
e = int(d)                 # ara e és un enter amb valor 123
print(e, type(e))
```

```
In [ ]: h = 3.14
g = int(h)                 # g és 3, la part decimal es perd
print(g, type(g))
```

```
In [ ]: j = 10
k = float(j)               # k és 10.0, conversió a decimal
print(k, type(k))
```

```
In [ ]: l = ""
m = bool(l) # f és False, perquè la cadena buida
print(m, type(m))

p = 0
q = bool(p) # q és False, perquè el valor és 0
print(m, type(m))
```

És important tenir en compte que no totes les conversions són possibles; per exemple, intentar convertir una cadena que no representa un número a enter generarà un error.

Gestió de memòria i variables en Python

Una variable és un nom simbòlic que fa referència a un objecte guardat en memòria (no conté directament el valor). Quan assignem una variable, el que fem que aquesta apunti a un objecte existent en memòria.

```
In [ ]: vara = 10
varb = vara
print(id(vara)) # Mostra la ubicació de l'objecte a la memòria
print(id(varb)) # Igual a id(a), perquè apunten al mateix objecte
```

```
varb = 20
print(id(varb)) # Diferent, ara 'b' apunta a un altre objecte
```

El `id()` és un identificador únic a la memòria.

Comptador de referències

És un mecanisme intern de Python que porta un registre del nombre de variables o referències que apunten a cada objecte en memòria. Quan s'assigna un objecte a una variable, aquest comptador s'incrementa, i quan la variable deixa de fer-hi referència, el comptador disminueix.

Quan arriba a zero, Python allibera automàticament la memòria que ocupava. Aquest procés s'anomena *garbage collector* i permet que els programadors no hagin de gestionar manualment la memòria.

D'aquesta manera Python manté la **memòria neta i eficient** durant l'execució dels programes.

Assignacions múltiples i tipatge dinàmic

Podem assignar el mateix valor a diverses variables a la vegada:

```
In [ ]: x1 = y1 = z1 = 20
print(id(x1), id(y1), id(z1)) # totes apunten al mateix objecte
```

Python és un llenguatge de tipatge dinàmic, per tant, el tipus d'una variable pot variar durant l'execució:

```
In [ ]: x2 = 10
print(type(x2)) # <class 'int'>
x2 = "hola"
print(type(x2)) # <class 'str'>
```

GIL (Global Interpreter Lock) i JIT

El **GIL (Global Interpreter Lock)** és un mecanisme intern de Python que assegura que només un fil (*thread*) pugui executar codi Python a la vegada.

Va ser creat per evitar problemes quan diversos fils accedeixen i modifiquen la mateixa memòria simultàniament, garantint així la seguretat de la gestió de memòria i la coherència de les dades.

El GIL limita la capacitat d'aprofitar processadors amb múltiples nuclis, ja que impedeix l'execució realment paral·lela de diversos fils en codi Python.

Aquest problema està sent abordat a partir de Python 3.13 amb l'entrada en escena d'una versió experimental anomenada "nogil", que elimina el GIL.

Aquesta nova versió utilitza un **compilador JIT (Just-In-Time)** que tradueix el codi Python en codi natiu optimitzat durant l'execució. Gràcies al JIT, l'execució sense GIL esdevé segura i eficient, permetent que diferents fils puguin executar codi Python en paral·lel de manera natural.

Aquest canvi important obre la porta a millors possibilitats de rendiment i escalabilitat

en aplicacions paral·leles, mantenint la gestió de memòria i variables tal com és, automàtica i senzilla per als desenvolupadors.

Algunes llibreries encara estan en procés d'adaptació a aquest nou model sense GIL, però suposa un avanç rellevant en l'evolució del llenguatge Python.